

Revision 1

From a prompt to a working RAG app — everything in Lectures 1–6, tied together.

By the end you can: explain an end-to-end RAG app architecture · debug a failing retrieval pipeline.

Press  for fullscreen · navigate with  

A grounded answer comes back from your RAG app. Put the **ONLINE** steps in the order they actually run: (i) embed the question (ii) LLM generates at temp=0 (iii) UI POSTs the question to the backend (iv) top-k chunks retrieved by cosine (v) assemble prompt = context + question.

iii → i → iv → v → ii ✓

i → iii → iv → ii → v

iii → iv → i → v → ii

i → iv → v → ii → iii

Six lectures, three blocks

- Block A — Talking to the model. L1 prompting & sampling · L2 structured output (JSON).
- Block B — Shipping it. L3 backend & REST · L4 frontend, reactivity & SSE streaming.
- Block C — Grounding it. L5 vector storage · L6 RAG.

Today

Recap each block, draw the end-to-end architecture, name the common mistakes, and quiz yourself — so you can build and debug a RAG app on your own.

The midterm — what it covers

Multiple-choice and very short answer, drawn from Lectures 1–6. It is **conceptual** — no coding — but you must know the vocabulary and why each piece exists. The next slides list every assessable outcome.

On the exam

- L1 prompting & the chat API
- L2 structured output & JSON
- L3 backend, REST & the data layer
- L4 frontend reactivity & SSE
- L5 embeddings & vector storage
- L6 retrieval-augmented generation

Know the terms cold

- Message roles · temperature · JSON vs schema mode
- Status codes (422 / 429 / 404) · router
→ service → repository · Alembic
- SSE (`text/event-stream` , `[DONE]`) · cosine similarity
- Retrieve-then-read · citations · hallucination

Assessable outcomes — L1 & L2

L1 — Prompting & the chat API

- `11m-1ec1-1` Use the message roles (system / user / assistant) correctly.
- `11m-1ec1-2` Control sampling with temperature for deterministic vs diverse output.
- `11m-1ec1-3` Apply few-shot and chain-of-thought prompting.
- `11m-1ec1-4` Explain what response streaming does (lowers perceived latency; does not speed generation).

L2 — Structured output & JSON

- `11m-1ec2-1` State what JSON mode guarantees (valid syntax) vs what it does not (the shape).
- `11m-1ec2-2` Use a Pydantic / JSON schema to constrain and validate the model's output.
- `11m-1ec2-3` Distinguish schema / type validation from semantic validation.

Assessable outcomes — L3 backend, REST & data

- `11m-1ec3-1` Choose HTTP verbs and status codes (POST for actions; 422 invalid body; 429 rate limit).
- `11m-1ec3-2` Model requests / responses with Pydantic and explain the automatic 422.
- `11m-1ec3-3` Explain the router → service → repository layering (what each layer does).

- `11m-1ec3-4` Explain the ORM and the repository as the only database-touching layer (SQLAlchemy).
- `11m-1ec3-5` Explain schema migrations with Alembic (versioned / ordered / reversible; upgrade head).
- `11m-1ec3-6` Explain why the API key stays server-side, and how a token-bucket rate limit works.

Assessable outcomes — L4 & L5

L4 — Frontend reactivity & SSE

- `11m-1ec4-1` Explain why Server-Sent Events (SSE) fit one-way token streaming over ordinary HTTP.
- `11m-1ec4-2` Identify the SSE format (text/event-stream) and the [DONE] sentinel.
- `11m-1ec4-3` Explain how a reactive UI updates (reassign state → re-render).

L5 — Embeddings & vector storage

- `11m-1ec5-1` Define an embedding (a fixed-length vector encoding meaning).
- `11m-1ec5-2` Interpret cosine similarity (+1 same, 0 unrelated, -1 opposite).
- `11m-1ec5-3` Explain the vector index (pgvector), nearest-neighbour search, and distance-vs-similarity ordering.
- `11m-1ec5-4` Explain chunking and why metadata is stored (citations, filtering).

Assessable outcomes — L6 RAG

- `11m-1ec6-1` Explain retrieve-then-read (the model only reads the top-k chunks).
- `11m-1ec6-2` Explain why RAG beats a plain prompt or fine-tuning for private, changing data.

- `11m-1ec6-3` Explain the `min_score` floor and the “I don't know” path.
- `11m-1ec6-4` Explain prompt assembly and citations (use only the context; say you don't know).

These four are the spine of the whole course — retrieve the right chunks, read *only* them, cite them, and refuse when they don't contain the answer.

Recap A — Prompting, sampling & JSON

The model (L1)

- A **next-token predictor**; stateless — you resend the whole history.
- Messages have roles: **system** (rules) · **user** · **assistant**.
- `temperature=0` for deterministic tasks; higher to be creative. Tune *temperature* or *top_p*, not both.
- Patterns: zero-shot · few-shot · chain-of-thought.

Missed it? Revisit L1/L2.

Structured output (L2)

- Ask for JSON, then **parse with** `json.loads` — never `eval`.
- **Validate** fields & types yourself: valid JSON ≠ valid data.
- On failure, run a **repair loop** — feed the error back to the model.
- `response_format` is **IGNORED** by the class **proxy** — prompt → parse → validate is the real path.

(L1) Your extractor returns the right facts but in a sloppy, inconsistent shape — sometimes a bulleted list, sometimes a paragraph. Which prompting pattern most reliably fixes the **FORMAT?**

Few-shot — show 2–5 examples of the exact output shape you want ✓

Chain-of-thought — ask it to reason step by step first

Raise temperature so it explores more formats

Add 'be consistent' to the system prompt and hope

On the class model (gemma-4-E4B-it via the proxy), you set `response_format={"type":"json_object"}` and still sometimes get prose around the JSON. Why, and what's the real fix?

JSON mode is silently ignored here — prompt for JSON, then `json.loads` + validate (and repair on failure)

✓

The temperature is too high — set it to 0 and JSON mode will be enforced

Use `eval()` instead of `json.loads` so Python is more lenient

Switch to a bigger model; only large models can emit JSON at all

Recap B — Backend, frontend & streaming

Backend & REST (L3)

- The LLM call lives **behind your server** — the API key never reaches the browser.
- Expose `POST /api/chat`; type the I/O with **Pydantic**.
- House style: **router** → **service** → **repository**. Handle errors; rate-limit.

Frontend & SSE (L4)

- **Reactivity**: state drives the UI; update state, the view re-renders.
- Stream tokens with **Server-Sent Events** (`text/event-stream`).
- Server `yield`s chunks; client reads the stream — lower *perceived* latency.

(L3) Your `/api/chat` sends well-formed JSON but omits the required ``messages`` field, and a load test fires 100 requests in a second.

Which status codes do you expect, respectively?

422 for the missing field (Pydantic rejected it), 429 for hitting the rate limit ✓

400 for the missing field, 500 for the rate limit

429 for the missing field, 422 for the rate limit

200 then 503 — FastAPI retries silently

(L4) Your SSE client occasionally JSON-parses a half-token and crashes. Network reads don't line up with SSE events. What's the correct read loop?

Accumulate bytes in a buffer, split on the blank line ``\n\n``, keep the trailing partial frame, and only parse complete events ✓

`json.loads` every chunk from `reader.read()` immediately — each read is exactly one event

Switch to a WebSocket; SSE can't stream reliably

Raise `max_tokens` so frames arrive whole

A teammate calls the LLM directly from the browser (fetch to the provider) and waits for the full reply before showing anything. Which TWO course rules does this break?

API key is exposed client-side (it belongs behind your backend) AND no streaming — stream tokens over SSE for lower perceived latency ✓

The temperature is wrong AND the chunks are too big

It needs a vector store AND a reranker

Nothing — calling the provider from the browser is the house style

Recap C — Vectors & RAG

Storage (L5) & RAG (L6)

- Embed text → **vectors**; rank by **cosine similarity**.
- **Chunk** documents (200–500 tokens, small overlap) + metadata.
- Index in a **vector store** (pgvector).
- RAG = **retrieve-then-read**: embed query → top-k → stuff into prompt → answer with citations.

Most "bad RAG" is bad **retrieval**, not a bad model — inspect what was retrieved first.

(warm-up) In one line: what does RAG actually do at query time?

Retrieve-then-read: embed the question, fetch the top-k relevant chunks, and stuff them into the prompt as context before the model answers ✓

Fine-tunes the model on your documents so the new facts are baked into its weights

Stores your whole corpus in the system prompt so the model always has every document in context

Lets the model browse the live web and cite whatever pages it finds

The debug ladder — a wrong RAG answer

Walk it **in order**. Look at the data at each boundary — don't start with the model.

1. **Was the gold chunk even retrieved?** Print the top-k. Missing → a *retrieval* problem, not the model.
2. **Is it in the index at all?** Stale index or bad chunking → it never had a chance. Re-index; check boundaries & overlap.
3. **Vocabulary mismatch?** Keyword misses synonyms ("car" vs "automobile") → switch to embeddings.
4. **Retrieved but ignored?** "Lost in the middle" → keep `k` small, best chunk first.
5. **Retrieval fine but it invents facts?** The *prompt* → "use ONLY the context", allow "I don't know",
confirm `temperature=0`.

The rule: inspect what was retrieved *before* you touch the model. Nine times out of ten the bug is visible in the chunks.

(L5) Cosine similarity ranks your retrieval hits. What does it measure, and what is its range?

The angle between two vectors: $+1$ = same direction, 0 = unrelated, -1 = opposite ✓

The Euclidean distance between vectors: 0 = identical, larger = less similar

The number of shared tokens: 0 to the vocabulary size

A probability from 0 to 1 that two texts are the same

(L6) Retrieval is correct — the gold chunk IS in your top-8 — but the model still ignores it. The chunk sits at position 5 of 8. Most likely cause and fix?

'Lost in the middle' — LLMs under-attend to the middle of long context; keep k small and put the best chunk first ✓

The embedding model is stale — re-index the whole corpus

Cosine similarity has a sign bug — flip the comparison

The rate limit dropped the chunk — retry with backoff

Spot the bug — the indexer

```
def build_index(doc):  
    chunks = chunk(doc,  
                    size=2000,  
                    overlap=0)  
  
    for ch in chunks:  
        v = embed(ch.text)  
        store.add(v, ch.text)  
  
    # ship it 
```

Two things are wrong here. Name them before you flip.

A whole-doc-ish vector is a *blurry average*; chunks pinpoint the passage. You can't cite what you didn't capture.

The bugs

- **size=2000** is too big — one vector blurs many ideas. Use **200–500 tokens**.
- **overlap=0** — a fact split across a boundary lands in neither chunk. Use **~10–20%**.
- **No metadata stored** — you can't cite source/page later.

One embedder, two paths

- Embed the **query** and the **documents** with the **same model** — always.
- Mix two models and the vectors live in *different spaces*; cosine similarity becomes meaningless noise.

On the ladder: a "space mismatch" bug — like a vocabulary mismatch, but the query and index can't even be compared.

Retrieval suddenly returns garbage after a teammate 're-indexed with a newer embedding model' but left the query path on the OLD model. Most likely cause?

Documents and queries are now embedded by DIFFERENT models — their vectors live in different spaces and aren't comparable ✓

The cosine similarity function has a sign bug

max_tokens is too low on the LLM call

The SSE stream dropped the citations

THINK · PAIR · SHARE — 3 MIN

Why does THIS prompt assembly reduce hallucination?

```
Answer using ONLY the context below.  
If the answer is not in the context,  
say you don't know.
```

```
[1] {chunk_1}
```

```
[2] {chunk_2}
```

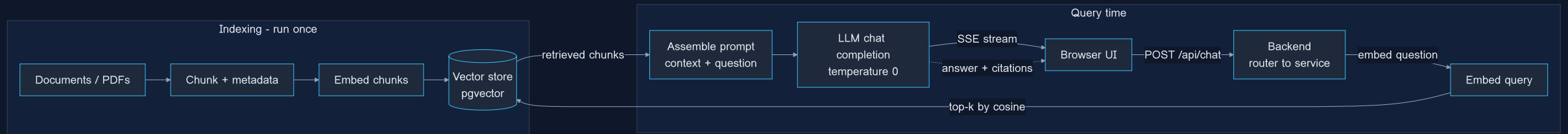
```
Q: {question}
```

Discuss each line:

- What does "**ONLY the context**" stop the model from doing?
- Why explicitly **allow "I don't know"**?
- Why **number** the chunks `[1] [2]`?
- What *else* belongs here for a production answer? (hint: `citations`, `temperature=0`)

Remember: the model never sees your whole corpus — only these top-k chunks. Quality is gated by retrieval first, the model second.

Architecture



Reading the architecture

Offline — build the index once

Load → chunk + metadata → embed each chunk → store the vectors. Do this *before* any question is asked; re-run when documents change.

Online — answer a question

UI calls your backend → embed the question → top-k from the store → assemble the prompt → LLM at `temperature=0` → stream back with citations.

Each lecture owns one box: L1–2 the prompt, L3–4 the backend + UI, L5 the store, L6 retrieve-then-read. The arrows are the app.

Common mistakes (and the fix)

- **Non-zero temperature** on a deterministic task
→ flaky output. Set `temperature=0`.
- **Parsing JSON by hand / `eval`** → crashes & RCE. `json.loads` + validate fields.
- **Bad chunking** (too big / too small / no overlap)
→ vague or split passages. 200–500 tokens, ~10–20% overlap.
- **Blaming the model** for bad answers → wrong fix. Inspect retrieved chunks first.
- **No "say I don't know"** → confident hallucination. "Use ONLY the context" + allow declining.

L2 caveat: `response_format` is *ignored* by the class proxy — `pydantic model_validate_json` is the validate path. **Block B:** `422` = bad fields, `429` = rate-limited; an SSE client must **BUFFER** partial frames until the blank line.

Rapid-fire — define it in one breath

Cover the right column. Say each definition out loud, then reveal.

- **Stateless** → no memory between calls; resend the whole history.
- **temperature=0** → (near-)deterministic; use for extraction / RAG.
- **Repair loop** → feed the JSON error back to the model and retry.
- **SSE** → `text/event-stream`; `data:` frames split by a blank line.
- **Cosine similarity** → angle between vectors; +1 same, 0 unrelated.
- **Chunk + overlap** → ~200–500 tokens, ~10–20% overlap, +metadata.
- **Retrieve-then-read** → embed query → top-k → assemble → generate.

Missed three or more? That block is your revision priority — go back to its lecture.

PREDICT → RUN → MODIFY

Before you run the toy RAG

The next slide runs the question `"what does the term code 1/2026 mean"` through our 4-chunk KB at $k=2$. Commit to an answer for each:

- (a) Does the gold chunk `h3` come back in the **top-2**?
- (b) What is the **2nd** retrieved chunk — and is it actually relevant?

Hint: the retriever is bag-of-words, so shared words like `"the"` count toward similarity. Write all three down, then run.

Toy Rag

```
# Consolidation: a whole RAG pipeline in pure Python.
# embed (bag-of-words) -> cosine top-k -> assemble the prompt you'd send the LLM.
# No libraries, no network: this is the shape of the real app from L5-L6.
import math

KB = [
    {"id": "h3", "text": "The term code 1/2026 means semester 1 of academic year 2026."},
    {"id": "s7", "text": "Students run Python in an in-browser sandbox powered by Piston."},
    {"id": "l6", "text": "RAG grounds answers in your own documents to reduce hallucination."},
    {"id": "x9", "text": "The cafeteria serves lunch from eleven until two each weekday."},
]

def tokens(t): return t.lower().replace(".", "").split()
```

Debug Broken Rag

```
# DEBUG DRILL (the named outcome, live): a RAG answer is wrong though the  
# fact IS in the corpus. Walk the L5-L6 debug ladder instead of guessing.
```

```
import math
```

```
KB = [  
    {"id": "h3", "text": "The term code 1/2026 means semester 1 of academic year  
2026."},  
    {"id": "s7", "text": "Students run Python in an in-browser sandbox powered by  
Piston."},  
    {"id": "l6", "text": "RAG grounds answers in your own documents to reduce  
hallucination."},  
    {"id": "x9", "text": "The cafeteria serves lunch from eleven until two each  
weekday."},  
]
```

```
def tokens(t): return t.lower().replace(".", "").split()  
vocab = sorted({w for c in KB for w in tokens(c["text"])})
```

(L6) A RAG answer is wrong even though the fact is in your corpus.
What should you check FIRST?

The model's temperature

Which chunks retrieval actually returned ✓

The SSE streaming code

The system prompt's persona

(L1) Your RAG endpoint returns a slightly different answer each run for the SAME question and SAME retrieved chunks. Most likely fix?

Set temperature=0 ✓

Increase max_tokens

Add more chunks to the context

Re-embed the corpus

Midterm practice — L1 & L2

Fresh questions (not on the exam). Cover the answer, decide, then check.

Q1 — roles. Dev A puts the app's rules in the `system` message; Dev B prepends them to every `user` message. Who is right, and why?

A. Dev A. Persona and rules belong in the `system` role — set once, apply to the whole conversation, not repeated per turn.

Q2 — validation. A reply is valid JSON and passes your Pydantic *types*, but sets `discount: 1.5` (150% off). Which validation layer must reject it?

A. Semantic (business-rule) validation. A schema checks shape and type; it cannot know 1.5 is out of range.

Midterm practice — L3 & L4

Fresh questions (not on the exam).

Q3 — migrations. You added a column to a SQLAlchemy model. In order, which two Alembic steps bring every database up to date?

A. (1) create a new migration script (`alembic revision --autogenerate`), then (2) `alembic upgrade head`. Migrations are ordered & reversible.

Q4 — SSE. Your endpoint sends tokens but the browser shows nothing until the very end. Which `Content-Type` does it need, and what terminates each SSE event?

A. `text/event-stream`; each event ends with a *blank line* (`\n\n`), which tells the browser to deliver it.

Midterm practice — L5 & L6

Fresh questions (not on the exam).

Q5 — cosine. A query and a chunk have cosine similarity ≈ 0 . How related are they, and what value would mean *opposite* meaning?

A. ≈ 0 means unrelated (orthogonal); -1 means opposite. $+1$ is identical direction.

Q6 — grounding. Your app confidently answers a question whose fact is *not* in the retrieved chunks. Name the failure and one fix.

A. Ungrounded *hallucination*. Fix: a `min_score` floor plus a prompt rule — “answer only from the context; if it isn't there, say you don't know.”

Two embedding vectors point in exactly opposite directions. Their cosine similarity is:

+1

0

-1 ✓

0.5

Showing the model 3 input → output examples so it infers the pattern is called:

Few-shot prompting ✓

Chain-of-thought

Fine-tuning

Zero-shot prompting

Enabling response streaming (stream=True) primarily improves:

Total generation speed

Perceived latency — tokens appear sooner ✓

The model's accuracy

The token cost

A provider's plain JSON mode is on. It guarantees the reply is:

Valid JSON syntax (but not your shape) ✓

Exactly your Pydantic schema

Free of hallucination

Always the same text

Your API allows 30 requests/min. A user's 31st request that minute arrives. What status code, and which header tells them when to retry?

429, with Retry-After ✓

503, with Location

403, with WWW-Authenticate

400, with Allow

Why does SSE fit LLM token streaming better than a plain JSON response?

It sends tokens as they're generated over one long-lived HTTP response ✓

It is encrypted by default

It uses less client memory

It guarantees valid JSON

For answering questions about your private, frequently-changing documents, RAG beats fine-tuning mainly because:

No retraining — just update the index, and it can cite sources ✓

It is cheaper to train

It needs no database

It removes hallucination entirely

EXIT CHECK — In your HW3 mini-RAG, an answer is confidently wrong though the fact IS in your corpus. Per the debug ladder, what is the **FIRST** thing you do?

Print the top-k chunks retrieval returned for that exact question ✓

Raise temperature so the model is more creative

Swap in a larger LLM

Add 'be accurate' to the system prompt

You can now

- Explain an **end-to-end RAG app**: prompt → backend → vector store → retrieve-then-read → streaming UI.
- **Debug a failing retrieval pipeline**: inspect the chunks, check chunking/overlap, walk the debug ladder.
- Avoid the classic traps: temperature, JSON validation, chunking, blaming the model.

Re-run the toy RAG, then revisit **HW3** — you now have the whole RAG block in one place to build and debug on your own.

Next lecture

Lecture 8

Evaluate RAG, post-processing & hybrid search — measure retrieval with recall@k / MRR, then improve it with reranking and keyword+vector search.

